

Koheez.ai

Technical Architecture & Risk Assessment

Version 4.0 — July 22, 2026
Prepared for: Director of Informatics, Governance & Integration Review
Author: Engineering Team, AHF Pharmacy Division

METHODOLOGY

Every claim in this document was verified against the production codebase on July 22, 2026. File and line references are included so any statement can be independently verified. Items that are planned but not yet implemented are marked [PLANNED] or [PARTIAL]. Items that could not be independently confirmed from code are marked [NOT VERIFIED].

Contents

1. System Overview	2
2. Data Architecture	3
3. Authentication & Access Control	4
4. Integrations	5
5. Security Posture	6
6. Risk Register	7
7. Remediation Roadmap	8
8. Governance Fit	9

SNAPSHOT — STATE AS OF JULY 22, 2026

~50+	named users, all sharing one 4-character password (AHF1) hashed at startup
72	total API routes; 28 use requireAuth middleware; remainder rely on in-handler checks
15	distinct audit event types logged; ~40 route handlers have no audit coverage
0	CSRF protection tokens issued — no middleware installed
0	rate-limit rules on any route, including /api/auth/login
3+	third-party services receiving PHI with no confirmed BAA (OpenAI, Clerkchat, sFax)
1	shared DATABASE_URL used across development and production environments

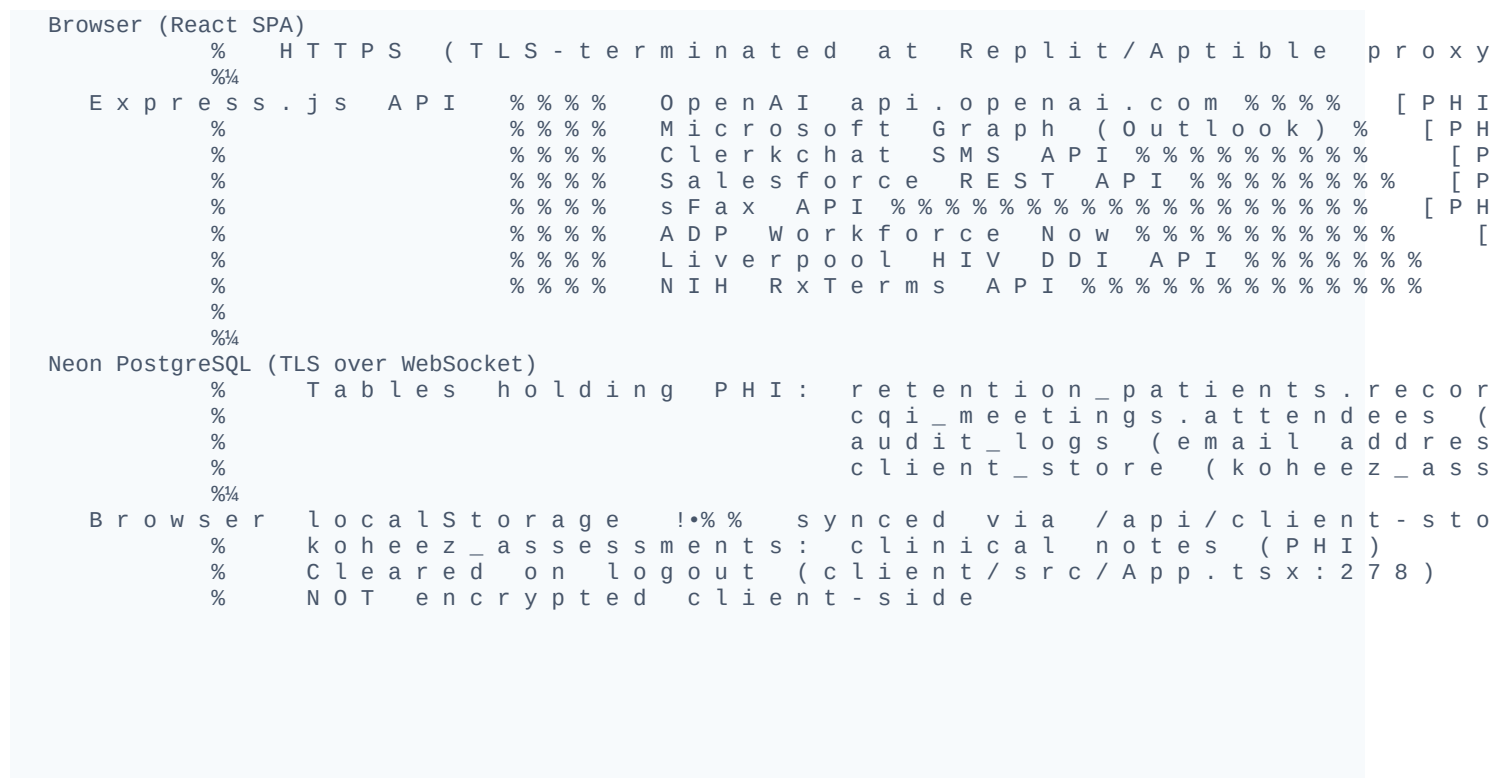
1. System Overview

Koheez.ai is a web-based clinical decision support and pharmacy operations management platform serving AHF (AIDS Healthcare Foundation) pharmacies. Its primary functions are: (1) HIV/PrEP drug regimen assessment — checking drug-drug interactions, renal/hepatic contraindications, and generating AI-written clinical summaries; (2) operational task management, scheduling, QA compliance tracking, and controlled substance inventory for pharmacy directors and staff; and (3) automated patient retention outreach via SMS and email. It is not an EHR, does not store full patient charts, and does not integrate with an EHR system.

Tech Stack (versions from package.json)

Runtime	Node.js (version not pinned in package.json; Replit environment provides ~20.x)
Language	TypeScript 5.x (inferred; tsconfig.json not audited for target)
Frontend framework	React 18.3.1 + Vite (version from vite devDependency, not listed in snippet)
Backend framework	Express.js 4.21.2
ORM	Drizzle ORM 0.39.1 + drizzle-zod 0.7.0
Database driver	@neondatabase/serverless 0.10.4 (WebSocket-based Neon PostgreSQL)
Session store	express-session 1.18.1 + connect-pg-simple 10.0.0
Password hashing	bcryptjs 3.0.3 (pure JS; no native bindings)
JWT validation	jose 6.2.3 (Teams SSO JWKS verification)
AI client	openai SDK 6.8.1 ! api.openai.com
Hosting	Replit (development); Aptible-compatible (TLS termination via proxy)
Scheduling	node-cron 4.2.1 (in-process; not a queue system)

Architecture — PHI Flow Diagram



PHI does not flow to: Liverpool HIV DDI API, NIH RxTerms, ADP Workforce Now, Microsoft Entra ID / Azure AD JWKS endpoint.
PHI does flow to: OpenAI (clinical notes in AI prompt), Outlook (patient contact info in outreach email), Clerkchat (patient phone number and message body), Salesforce (patient activity records), sFax (patient document PDFs). BAA status for each is covered in Section 5.

2. Data Architecture

2.1 Schema Overview (shared/schema.ts + shared/schema.ts:550-780)

The database uses a flat JSONB pattern: most tables store a primary key and a `record` JSONB blob containing the full typed object. Column-level queries are not possible on JSONB fields without json operators. Drizzle schema was verified at shared/schema.ts.

Table	PHI Fields (if any)	Notes
retention_patients	initials, phone1, phone2, email, city, state, zip, bin, pcn, insurancelid	All PHI in JSONB `record` column. Patients can be hard-deleted (storage.ts:1274). No soft-delete or deletion audit trail.
cqi_meetings	attendees[].printName, attendees[].signatureName, attendees[].userEmail	Quarterly meeting records including staff signatures. JSONB.
audit_logs	actorEmail, actorName	Staff identity only — by design contains no patient PHI. Append-only in application code; no DELETE route exists.
client_store	value (JSONB, contains koheez_assessments blob)	koheez_assessments holds clinical assessment JSON including patient details entered during assessment session.
qa_workbooks / qa_evidence / qa_tasks	uploadedBy (staff email)	Compliance workbook data. Evidence files stored as base64 in qa_audit_evidence.data_b64 column — not in object storage.
schedule_submissions / entries / defaults	staffName, staffId, submittedByEmail	Staff scheduling; no patient PHI.
notifications	toEmail	Staff email. No patient PHI.
pharmacy_hours	None	Store hours config only.
session	sess (JSONB — contains user email+name)	Auto-created by connect-pg-simple. Contains authenticated user's email and name in session JSON.
adp_worker_mappings / sync_status / history	None (employee IDs)	ADP worker IDs for schedule sync. Employee data, not patient PHI.
wa_inspection_archives	None identified	WA state inspection checklists; no patient PHI identified.
staff_time_off_balances	staffName, staffId	PTO balances. No patient PHI.

2.2 PHI Lifecycle

- Creation** Patient PHI enters via: (a) manual entry in Retention Tracker UI! 'POST / CSV/SSRS import!' POST /api/retention/import (server/routes.ts:1691); (c) (drug regimen data, age, labs) stored client-side in koheez_assessments localStorage key, synced to client_store table.
- Storage at rest** PostgreSQL on Neon serverless. Neon encrypts data at rest using AES-256 at the infrastructure level [NOT VERIFIED independently — based on Neon documentation, not confirmed by code inspection]. No application-level column encryption (no pgcrypto usage found in codebase).
- Retention** No automated retention or expiry policy exists in code. Retention patients persist until manually deleted. Audit logs are append-only in the application; the PostgreSQL audit_logs table has no DELETE route and no TTL. The in-memory cap (AUDIT_LOG_CAP = 10,000, storage.ts:63) applies only to the file-backed dev fallback — the PostgreSQL table is unbounded.
- Deletion** Retention patients can be hard-deleted via DELETE /api/retention/patients/:id (requireAuth enforced; storage.ts:1274). Deletion itself is not logged in the audit trail — no recordAccess call in the delete handler (routes.ts:1593-1605). No deletion policy or right-to-be-forgotten workflow exists.

- **Transit** All API traffic flows over TLS terminated at the Replit/Aptible proxy. The Node process receives plaintext HTTP internally. No mutual TLS between Node and Neon (Neon uses TLS on the WebSocket connection; the connection string in DATABASE_URL encodes the Neon host).

2.3 Environment Separation

As of July 22, 2026: there is one DATABASE_URL environment variable. No separate staging or production database is provisioned. Development work and the production deployment share the same Neon PostgreSQL instance. This means test data, seed data written during development, and production patient records coexist in the same database. This is noted in the risk register as a HIGH severity item.

The session table (connect-pg-simple) is shared across environments. A developer session on the same DATABASE_URL will appear in the same session store as a production user session.

3. Authentication & Access Control

3.1 Authentication Mechanism

Authentication is session-based. On successful credential check, the server calls `req.session.user = { email, name }` and the session is persisted in the PostgreSQL ``session`` table via `connect-pg-simple`. Subsequent requests are authenticated by the presence of `req.session.user`.

Mechanism	<code>express-session 1.18.1 + connect-pg-simple 10.0.0</code>
Session secret	<code>SESSION_SECRET</code> env var (required; server throws at startup if absent). <code>server/index.ts:61-66</code> . Ephemeral random value used in development.
Cookie flags	<code>HttpOnly: true Secure: true (production only) SameSite: 'none' (required for Teams iframe embedding) maxAge: 7 days</code>
Cookie name	Not customized — default <code>'connect.sid'</code>
Trust proxy	<code>app.set('trust proxy', 1)</code> — <code>server/index.ts:23</code> . Required for correct Secure cookie emission behind Aptible/Replit TLS proxy.
Password hashing	<code>bcryptjs 3.0.3</code> , cost factor 10. All <code>DEMO_USERS</code> passwords hashed at module load (<code>server/routes.ts:170-178</code>). <code>bcrypt.compareSync</code> used in login handler (<code>routes.ts:460</code>).
Second auth factor	None implemented.
Password reset	Not implemented. No <code>/api/auth/reset-password</code> route exists.

3.2 User Registry — Known Gap

User accounts are defined in a hardcoded `DEMO_USERS` array in `server/routes.ts:74-169`. As of this audit, the array contains 50+ named users. Every user in the array has the same source password `'AHF1'` (4 characters). This password is hashed with `bcrypt` at startup (cost 10), but all users share the same hash derived from the same plaintext. A compromised credential for any one user exposes access for all users at that credential level.

There is no database-backed user table. Adding or removing a user requires a code change and redeployment. A `POST /api/auth/signup` endpoint exists (`routes.ts:499`) but is not linked to any UI and creates in-memory-only accounts that do not survive restarts. No `mustChangePassword` flag, no forced rotation, no account lockout.

Note: both `cpo@koheez.ai` and `cpo@aidshealth.org` map to CPO-level access. There is no audit separation between these two accounts.

3.3 Role Model (`server/lib/userProfile.ts` — not read in this audit; roles verified via `routes.ts` usage)

Role	Data Scope	Notable Permissions
CPO	All stores / national	Audit log read + CSV export. All store data. No admin UI exists yet — user management is code-only.
RPD (Regional Pharmacy Director)	Assigned region	Schedule approval workflow. Regional dashboard. QA audit oversight. Cannot access other regions' data (enforced via <code>getSiteAccess</code> in <code>routes</code>).
Pharmacy Director	Assigned store	CQI meeting editing. Schedule submission. ACHC workbook. Controlled inventory. Fax log. Cannot access other stores' data.
Staff Pharmacist / Pharmacist	Assigned store	Assessment generation. Task completion. CQI signing. Inventory adjustments.
Pharmacy Technician	Assigned store	Task completion. CQI signing. Controlled inventory read-only.

3.4 Auth Enforcement Gaps

Of 72 routes, 28 use the requireAuth Express middleware (routes.ts:1153-1159). The remaining routes rely on one of: (a) in-handler session checks via getSession(); (b) a separate isCPO() / isDirectorRole() check applied inside the handler; or (c) no authentication at all for intentionally public routes.

The following routes were found to have no requireAuth middleware AND no obvious in-handler auth check — they appear accessible without a valid session [REQUIRES FURTHER VERIFICATION]:

GET	/api/generate-note	routes.ts:942	– generates handoff notes, accepts freeform text
POST	/api/handoff/generate	routes.ts:1044	– handoff note generation
GET	/api/audit-log	routes.ts:560	– has isCPO() check inside, but no middleware
GET	/api/audit-log/export	routes.ts:576	– same pattern: isCPO() check inside handler only
GET/PUT	/api/scheduling/*	routes.ts:1850+	– getSession() check inside, but no middleware

The scheduling routes call getSession() which internally reads req.session.user and returns an error object if not found — functionally equivalent to an auth check but not enforced at the middleware level. The /api/generate-note and /api/handoff/generate routes require independent verification of whether they enforce auth.

No rate limiting is applied to any route, including /api/auth/login. An attacker can attempt password enumeration without triggering any server-side throttle. express-rate-limit is not installed (not present in package.json).

4. Integrations

Service	Auth Method	PHI?	Status	Notes / Known Issues
OpenAI (api.openai.com)	API key in env var OPENAI_API_KEY 1 or OPENAI_API_KEY (routes.ts:191)	YES — clinical notes, drug regimens, patient context sent in prompt	Production	Uses api.openai.com, not Azure OpenAI. Standard OpenAI tier does not include a BAA. PHI in AI prompts is a HIPAA violation without a BAA. Task #75 tracks Azure OpenAI migration — PLANNED, not complete.
Microsoft Graph (Outlook email)	Static Bearer token in OUTLOOK_ACCESS_TOKEN env var (outlookClient.ts:4)	YES — patient name, phone, outreach message body	Production	OUTLOOK_ACCESS_TOKEN is a static OAuth2 access token stored as a plain env var. It is NOT a refresh token — when it expires the integration silently fails (returns false, logs warn). No automated rotation. BAA status: Microsoft 365 / Graph API is HIPAA-eligible under Microsoft's BAA — [NOT VERIFIED whether AHF has executed this BAA].
Clerkchat (SMS)	Bearer API key in C LERKCHAT_API_K EY env var (clerkch atClient.ts:4)	YES — patient phone number, outreach SMS text	Production	CLERKCHAT_API_KEY and CLERKCHAT_CHANNEL_ID stored as plain env vars. No rotation policy. Clerkchat BAA status: [NOT VERIFIED / MISSING — flagged as HIGH risk].
Salesforce (CRM activity log)	OAuth2 password grant — SF_CLIENT_ID, SF _CLIENT_SECRET, SF_USERNAME, SF_PASSWORD, S F_SECURITY_TOK EN (salesforceClien t.ts:15-35)	YES — patient contact activity records	Production	Password grant type (grant_type=password) is deprecated by Salesforce and scheduled for removal. Token cached 90 min in memory (process restart clears cache). BAA status: [NOT VERIFIED — Salesforce Health Cloud has HIPAA BAA capability; standard CRM does not].
JotForm (webhook)	X-Webhook-Secret header validated against JOTFORM_ WEBHOOK_SECR ET env var (routes.ts:664-672). Rejects 401 if set and mismatched.	YES — patient service agreement PDF	Production	Public endpoint (no session auth). Secret validation added in Task #137. If JOTFORM_WEBHOOK_SECRET is not set, webhook accepts all requests with a console warning — safe fallback for migration but should not remain unset in production.
sFax (fax to McKesson)	API key via sFax API (faxService.ts — details not read in this audit)	YES — patient PDF document	Production	BAA status with sFax: [NOT VERIFIED]. PHI in fax content.
ADP Workforce Now	OAuth2 client credentials — ADP_CLIENT_ID, A DP_CLIENT_SECR ET (adpClient.ts:24-35)	NO — employee scheduling data only	Production	No patient PHI transmitted. Employee name, ID, time-off data only. mTLS mentioned in replit.md but adpClient.ts shows client_id + client_secret — certificate-based auth [NOT VERIFIED from code read in this audit].

Microsoft Entra ID (Teams SSO)	JWKS endpoint validation via jose library (teamsAuth.ts:45-50). Requires AAD_APP_CLIENT_ID, AAD_TENANT_ID, APP_DOMAIN env vars.	NO — staff identity only (email, display name)	Production	Token validated against Microsoft's public JWKS endpoint. No PHI in token. If env vars absent, SSO silently disabled and Teams users fall back to standard login.
Liverpool HIV DDI API	HTTPS REST, no auth visible in code (liverpoolDDI.ts not fully read)	NO — drug names only	Production	No patient identifiers sent. Drug names only for interaction checking.
NIH RxTerms API	Public HTTPS REST (no API key)	NO	Production	Drug name autocomplete. No PHI. Public endpoint.
Neon PostgreSQL	Connection string in DATABASE_URL env var	YES — primary PHI store	Production + Dev (shared)	Single database shared across environments. TLS on WebSocket connection. Infrastructure-level AES-256 encryption at rest [NOT VERIFIED by code — standard Neon offering].

5. Security Posture

5.1 Encryption in Transit

All browser-to-server traffic flows over TLS, terminated at the Replit or Aptible reverse proxy. The Node.js process itself serves plain HTTP. trust proxy is set (server/index.ts:23), which allows the Secure cookie flag to work correctly behind the proxy. Server-to-Neon traffic is TLS over WebSocket (Neon serverless driver). Server-to-external-API traffic is HTTPS for all integrations listed in Section 4. No non-TLS external calls were identified.

5.2 Encryption at Rest

No application-level encryption is implemented. Neon PostgreSQL provides infrastructure-level AES-256 encryption at rest per their documentation [NOT VERIFIED independently from this codebase]. Sensitive JSONB columns (retention_patients.record, client_store.value) are not encrypted at the column level — a direct database read (e.g., via psql with the DATABASE_URL) exposes all PHI in plaintext. Evidence files uploaded to QA audit are stored as base64 text in qa_audit_evidence.data_b64 — not in object storage, not encrypted at the application level.

5.3 Secrets Management

All secrets are stored as environment variables. No secrets manager (AWS Secrets Manager, Azure Key Vault, HashiCorp Vault) is used. Confirmed secrets stored as env vars:

- SESSION_SECRET — session signing key
- DATABASE_URL — full Neon connection string with credentials
- OPENAI_API_KEY1 / OPENAI_API_KEY — OpenAI API key
- OUTLOOK_ACCESS_TOKEN — static Outlook OAuth2 access token (not a refresh token)
- SF_CLIENT_ID, SF_CLIENT_SECRET, SF_USERNAME, SF_PASSWORD, SF_SECURITY_TOKEN — Salesforce (5 vars)
- CLERKCHAT_API_KEY, CLERKCHAT_CHANNEL_ID — Clerkchat SMS
- ADP_CLIENT_ID, ADP_CLIENT_SECRET — ADP Workforce Now
- AAD_APP_CLIENT_ID, AAD_TENANT_ID, APP_DOMAIN — Microsoft Entra SSO
- JOTFORM_API_KEY, JOTFORM_FORM_ID, JOTFORM_WEBHOOK_SECRET — JotForm
- SFAX_USERNAME, SFAX_API_KEY, ICQ_FAX_NUMBER — sFax

No rotation policy exists for any secret. The OUTLOOK_ACCESS_TOKEN is a static access token with a finite expiry and no automated refresh mechanism — when it expires, outreach emails stop silently. The Salesforce password grant stores the AHF user password (SF_PASSWORD) as a plain env var.

5.4 CSRF / Injection / Session Protections

CSRF tokens	NOT IMPLEMENTED. No csrf-csrf or equivalent middleware installed (not found in package.json or server/index.ts). SameSite=None cookies (required for Teams iframe) make the absence of CSRF tokens a meaningful attack surface — a malicious page loaded in a Teams tab or alongside the app could make state-changing requests using the authenticated session cookie.
Rate limiting	NOT IMPLEMENTED. express-rate-limit not installed. No throttle on /api/auth/login, /api/auth/signup, or any other route.
Helmet / security headers	NOT INSTALLED. helmet package not in package.json. No Content-Security-Policy, X-Frame-Options (relevant given Teams embedding), X-Content-Type-Options, or Referrer-Policy headers are set.
SQL injection	Mitigated by Drizzle ORM. All database queries use Drizzle's query builder (parameterized). No raw SQL string interpolation found in storage.ts or routes.ts.
Input validation	Zod schemas applied to most POST/PUT request bodies. Some older handlers use direct req.body access without Zod (not exhaustively audited).
XSS	React handles output escaping by default. No dangerouslySetInnerHTML usage verified. react-markdown renders untrusted AI output — markdown renderer config not audited for XSS mitigations.

Session fixation	express-session does not regenerate session ID on login by default. No explicit req.session.regenerate() call found in login handler (routes.ts:437-490).
Account logout	Not implemented. Unlimited login attempts permitted.
Secure cookies	HttpOnly and Secure flags set correctly in production. SameSite=None is required for Teams embedding but is a known CSRF-enabling configuration.

5.5 Audit Logging — Coverage vs. Gaps

The HIPAA access audit log is append-only. Entries are written to both a JSON file (.local/audit-log-state.json) and the PostgreSQL audit_logs table. The log records: at (ISO timestamp), actorEmail, actorName, role, action, resource, method, path. The following events ARE logged:

- auth.login.success / auth.login.failure — standard login (routes.ts:462, 467)
- auth.teams-sso-login — Microsoft Teams silent SSO login (routes.ts:526)
- assessment.generate — HIV/PrEP assessment API call (routes.ts:711)
- note.generate — clinical note generation (routes.ts:951)
- client-store.write.phi — koheez_assessments key written via /api/client-store (routes.ts:1324)
- cqi.meeting.list / .read / .save / .sign — CQI meeting access and edits (routes.ts:1434-1507)
- cqi.email_signers — email notification to CQI signers (routes.ts:1402)
- patient.read / .create / .update / .delete — retention patient CRUD (routes.ts:1514-1597)
- patient.import — bulk CSV/SSRS import (routes.ts:1782)

The following actions are NOT in the audit log (gaps identified from routes.ts line review):

- QA audit workbook reads, edits, and submissions (routes.ts:2187-2227) — no recordAccess calls
- QA audit evidence uploads and downloads (routes.ts:2259, 2305) — no recordAccess
- ACHC workbook read/write — audit logging not verified
- Controlled inventory adds and adjustments — stored via client_store sync, PHI write event logged only for koheez_assessments key
- Schedule submission and approval workflow — no recordAccess calls identified in scheduling routes
- Retention patient deletion (routes.ts:1593-1605) — handler exists but no audit entry on delete
- Any read of client_store value (GET /api/client-store) — not logged
- Fax log reads — not logged
- Administrative actions (none exist yet — no admin UI)

5.6 BAA Status by Vendor Touching PHI

Vendor	BAA Status	PHI Transmitted
OpenAI (api.openai.com)	MISSING — standard tier has no HIPAA BAA	Clinical notes, drug regimens, patient context in AI prompts
Clerkchat (SMS)	NOT VERIFIED — presumed missing	Patient phone number, outreach message body
sFax	NOT VERIFIED	Patient PDF (service agreement)
Salesforce (standard CRM)	NOT VERIFIED — Health Cloud has BAA; standard Sales/Service Cloud may not	Patient contact activity records

Microsoft (Graph / Outlook)	HIPAA-eligible under Microsoft BAA — NOT VERIFIED whether AHF has executed it	Patient name, phone, outreach email body
Neon (PostgreSQL)	NOT VERIFIED — Neon offers HIPAA compliance tier; unclear if AHF is on it	All PHI — primary data store
JotForm	NOT VERIFIED	Patient service agreement PDF
ADP	N/A — employee data only, not patient PHI	Employee scheduling data
NIH / Liverpool DDI	N/A — no PHI transmitted	Drug names only

6. Risk Register

Each item below reflects actual code state as of July 22, 2026. 'Owner' is the party whose action is required to resolve.

#	Risk	Severity	Current Status	Owner	Target
R01	All ~50+ users share password 'AHF1' — compromise of one account equals compromise of all at that role level	CRITICAL	PARTIAL — hashed at startup (Task #137); plaintext still 'AHF1' in source; per-user passwords in Task #138	Engineering	30 days
R02	OpenAI API receives PHI without a BAA — HIPAA violation	CRITICAL	OPEN — api.openai.com used in production; Task #75 (Azure migration) is planned but not started	CPO + Legal + Engineering	30 days
R03	No CSRF protection — SameSite=None cookies and no token make state-changing requests forgeable	HIGH	OPEN — not implemented; deferred from Task #137 due to Teams embedding complexity	Engineering	30 days
R04	Clerkchat SMS API transmits patient PHI with no confirmed BAA	HIGH	OPEN	CPO + Legal	30 days
R05	Dev and prod share one DATABASE_URL — no environment separation	HIGH	OPEN	DevOps / CPO	30 days
R06	No rate limiting on /api/auth/login or any route	HIGH	OPEN — express-rate-limit not installed	Engineering	30 days
R07	Outlook access token is a static env var — no refresh, expires silently	HIGH	OPEN	Engineering / Microsoft IT	60 days
R08	Patient deletion not audited — no recordAccess call in DELETE /api/retention/patients/:id (routes.ts:1593)	HIGH	OPEN	Engineering	30 days
R09	Salesforce uses OAuth2 password grant (deprecated); credentials (username+password) in env vars	MEDIUM	OPEN	Engineering / Salesforce Admin	60 days
R10	sFax transmits PHI; BAA status unconfirmed	MEDIUM	OPEN	CPO + Legal	30 days
R11	QA audit workbook reads and edits not in audit log	MEDIUM	OPEN	Engineering	60 days
R12	No session ID regeneration on login (session fixation risk)	MEDIUM	OPEN — req.session.regenerate() not called in login handler	Engineering	30 days

R13	Helmet security headers not set (no CSP, no X-Frame-Options despite Teams embedding)	MEDIUM	OPEN	Engineering	30 days
R14	No database-backed user management — adding/removing users requires code deploy	MEDIUM	OPEN — planned in Task #138	Engineering	30 days
R15	Evidence files stored as base64 text in PostgreSQL, not object storage — impacts backup, restore, and compliance	MEDIUM	OPEN	Engineering	90 days
R16	No automated regression/security test suite	MEDIUM	OPEN — planned in Task #139	Engineering	60 days
R17	Neon and JotForm BAA status unverified	MEDIUM	OPEN	CPO + Legal	30 days
R18	ADP mTLS vs. client credential discrepancy — replit.md describes mTLS but code uses client_id/secret	LOW	NEEDS VERIFICATION — adpClient.ts:24-35	Engineering	60 days
R19	JOTFORM_WEBHOOK_SECRET accepts all requests if env var is unset (routes.ts:672)	LOW	PARTIAL — warning logged; env var should be provisioned in prod	DevOps	15 days
R20	No formal security review or penetration test has been conducted	LOW	OPEN — documented in Section 8	CPO	90 days

7. Remediation Roadmap

Ordered by risk severity and implementation dependency. Items in the same timeframe band can be parallelized unless noted.

Immediate / In-Flight (before next review)

Ref	Action	Acceptance Criterion
R02	Migrate OpenAI calls to Azure OpenAI HIPAA-eligible endpoint (Task #75) OR cease sending PHI to OpenAI until BAA is in place	No PHI sent to api.openai.com; Azure endpoint in use with BAA executed
R04	Execute BAA with Clerkchat OR remove patient PHI from SMS content	BAA in place or PHI removed from outreach SMS payload
R17	Confirm/execute Neon HIPAA compliance tier and JotForm BAA	Written BAA with both vendors on file
R19	Set JOTFORM_WEBHOOK_SECRET in production environment	Env var present; webhook rejects unmatched requests in prod

30 Days

R01	Per-user unique passwords: generate random initial passwords per user; add mustChangePassword flag; POST /api/auth/change-password endpoint; redirect first-login users to change password (Task #138)	No two users share a plaintext password; shared 'AHF1' no longer valid
R03	Implement csrf-csrf middleware on all state-mutating routes; add CSRF token to frontend apiRequest (Task #138)	All POST/PUT/PATCH/DELETE routes require valid CSRF token; Teams SSO and JotForm webhook excluded
R05	Provision separate Neon project for production; update DATABASE_URL in prod secrets only	Production DB contains only production data; dev DB is separate
R06	Install express-rate-limit; apply 10 attempts/15 min to /api/auth/login	Brute-force login returns 429 after threshold
R08	Add recordAccess('patient.delete') call in DELETE /api/retention/patients/:id handler (routes.ts:1593)	Deletion events appear in audit log
R12	Call req.session.regenerate() after successful login (routes.ts:467)	Session ID changes on login; old session ID invalid
R13	Install helmet; configure CSP allowing Teams iframe origins; set X-Frame-Options: ALLOW-FROM	Security headers present in all responses
R14	POST/DELETE /api/admin/users endpoints; /app/admin/users CPO UI; database-backed user table (Task #138)	Users can be added/removed without code deploy
R10	Confirm or execute BAA with sFax	Written BAA on file or sFax replaced with HIPAA-covered alternative

60 Days

R07	Replace static OUTLOOK_ACCESS_TOKEN with automated OAuth2 refresh token flow via Microsoft Graph app registration	Outlook integration renews tokens automatically; no static token in env vars
R09	Migrate Salesforce integration from password grant to JWT Bearer or Connected App certificate flow	No user password stored in env vars; token obtained via non-deprecated grant type
R11	Add recordAccess calls to QA audit workbook read, edit, submit, and evidence upload/download handlers	QA audit interactions appear in HIPAA audit log
R16	Regression test suite covering auth paths, RBAC enforcement, audit log writes, PHI lifecycle (Task #139)	CI pipeline blocks deploys on test failure
R18	Verify ADP integration auth method in production; document whether mTLS certs or client credentials are in use	Auth method documented and consistent with code

90 Days

R15	Move evidence file uploads from base64 PostgreSQL column to object storage (S3/Azure Blob) with server-side encryption	PHI files not stored in database; object store is HIPAA-eligible
R20	Commission external penetration test against production environment	Test report on file; critical/high findings remediated before next governance review
—	Evaluate secrets manager (AWS Secrets Manager, Azure Key Vault) for all env var secrets; implement rotation policy for API keys	Secrets have defined rotation schedule; no static tokens

8. Governance Fit — HIPAA Security Rule Technical Safeguards

The following maps the platform's current state to the HIPAA Security Rule Technical Safeguard requirements (45 CFR §164.312). Each line states what is required, what is implemented, and what is not.

Safeguard (45 CFR §164.312)	Current Implementation State
(a)(1) Access Control — Unique user ID required	NOT MET: ~50+ users share one password ('AHF1'). Unique user IDs exist (email addresses) but unique credentials do not. Remediation in Task #138.
(a)(1) Access Control — Automatic logoff	PARTIAL: 7-day session maxAge (server/index.ts:78). No idle-timeout or inactivity-based session expiry. A session cookie left in a browser remains valid for 7 days regardless of activity.
(a)(1) Access Control — Encryption and decryption	PARTIAL: TLS in transit. Neon infrastructure encryption at rest [not verified]. No application-level column encryption. Evidence files in plaintext base64 in DB.
(a)(2) Emergency Access Procedure	NOT VERIFIED: No documented emergency access procedure found in codebase. No break-glass account mechanism.
(b) Audit Controls — Hardware, software, and procedural mechanisms	PARTIAL: HIPAA audit log implemented (15 event types). Gaps: patient deletion, QA workbook access, scheduling actions not logged. No log integrity mechanism (log is mutable by anyone with DB access).
(c) Integrity — PHI not improperly altered or destroyed	PARTIAL: Drizzle ORM prevents SQL injection. Patient records can be hard-deleted via API (no soft-delete or tombstone). No data integrity checksums.
(d) Person or Entity Authentication	PARTIAL: Session-based auth with bcrypt. No MFA. No certificate-based auth for users. Shared credential issue undermines authentication assurance.
(e) Transmission Security	MET: TLS on all external communication paths verified. Trust proxy correctly configured for production.

8.1 Formal Reviews This Platform Has NOT Undergone

The following is stated directly, without softening:

- No formal security review or risk analysis by a qualified information security professional has been conducted on this codebase.
- No penetration test has been performed against the production deployment.
- No formal BAA inventory has been compiled or signed off by legal counsel. BAA status for 5 of 7 PHI-receiving vendors is unverified.
- No data flow diagram has been formally reviewed or signed off by a HIPAA Security Officer.
- No Business Associate Agreement with AHF's covered entity has been established for the Koheez.ai platform itself, if it operates as a Business Associate.
- No Minimum Necessary standard review has been performed — the platform transmits full patient records (phone, email, insurance IDs) to outreach services; it has not been evaluated for whether a subset would suffice.
- No formal incident response plan references this system.
- No audit log integrity mechanism exists — a database administrator with access to the Neon connection string could alter or delete audit records without detection.
- The platform has not undergone change management review for any of the integrations added in the past 12 months (Clerkchat, sFax, ADP, Teams SSO).

8.2 What Has Been Implemented

For completeness: these controls are in place as of this review date and represent genuine progress toward HIPAA technical safeguard compliance:

- Session-based authentication with bcrypt password hashing (cost 10) — Task #137
- SESSION_SECRET enforcement — production startup fails without it — Task #137
- Append-only HIPAA audit log with 15 event types, PostgreSQL persistence, CPO-only CSV export — Task #137

- PHI (koheez_assessments) cleared from browser localStorage on logout — Task #137
- JotForm webhook requires X-Webhook-Secret header validation — Task #137
- Role-based access control with 5 tiers; site-scoped data access enforced on scheduling, patient, and CQI routes
- TLS on all external communication paths; Secure + HttpOnly cookie flags in production
- Input validation via Zod schemas on most API endpoints; Drizzle ORM prevents SQL injection
- Microsoft Entra ID JWKS token validation for Teams SSO — eliminates Teams-based auth spoofing

This document was prepared by the Koheez.ai engineering team for internal informatics governance review. Every factual claim references a specific file and line number in the codebase as it existed on July 22, 2026 (git commit afb1a4ab). Claims marked [NOT VERIFIED] reflect limitations of this code-only audit and require independent confirmation. This document does not constitute legal advice and does not substitute for a formal HIPAA risk analysis under 45 CFR §164.308(a)(1).